

LLMPAGER V0.13.0

One Trillion Parameters. One 16-Gigabyte GPU.

Kimi K2.6 – a 1T-parameter mixture-of-experts model – generating coherent text on a consumer RTX 5060 Ti, with 571GB of experts streaming from NVMe.

Glenn West · August 2026

The headline numbers

1T

total parameters (Kimi K2.6)

571_{GB}

of experts on NVMe

16_{GB}

of VRAM – 36× smaller

829_{GB}

streamed for the first 60 tokens

2

days from git init to 1T tokens

\$429

the GPU's street price

In plain terms: the model is thirty-six times bigger than the card's memory. It runs anyway.

Why this shouldn't work

- Conventional serving loads **every weight into GPU memory**. A 1T model at 4-bit needs ~570GB – that's seven H200s, roughly a quarter-million dollars of hardware.
- CPU offload helps until the model outgrows RAM too. This box has 128GB – still 4.5× too small.
- But a mixture-of-experts model has a secret: **each token only activates ~3% of the weights**. Kimi picks 8 of 384 experts per layer, per token.

The bet: if the working set is small and skewed, treat the GPU like a CPU treats its cache – and treat the SSD like memory. Classic virtual-memory paging, applied to a trillion parameters.

The **architecture** (unchanged since day one)

```
token → router (resident, VRAM)
      | top-8 expert ids, per layer
      ▼
ExpertCache[layer] —hit→ weights already in VRAM
                  | miss
                  ▼
Pager: 0_DIRECT read (NVMe) → host RAM tier → pinned → VRAM
      io worker pool · CUDA events gate compute · LFU eviction
```

- Resident core: attention, router, norms – plus the KV cache.
- Experts live in a 4096-aligned pack on disk; the hot ones are cached in VRAM.
- The same engine that runs Qwen3-30B at 41 tok/s runs Kimi at 1T – **only the scale changed**.

What Kimi added to the problem

Dimension	Qwen3-30B-A3B	Kimi K2.6
Total / active parameters	30B / 3B	1T / 32B
Experts	128 × 48 layers	384 × 61 layers
Expert pack	15.4 GB	570.9 GB
Attention	GQA	MLA (latent, compressed)
Router	softmax top-k	sigmoid + bias correction
Extras	–	shared expert, dense first layer, YaRN 256K

MLA is the gift: the whole KV state is 2.3KB per token – a quarter-million-token context fits in half a gigabyte. The experts are the entire challenge.

MLA: attention through a **keyhole**

- DeepSeek-style latent attention compresses each token's K and V into one **576-float vector**, shared by all 64 heads.
- Per-head keys and values are **never materialized** at decode: queries are absorbed through W_{kv}^T once per token, outputs reconstructed after the softmax.
- Cost per token: two small batched GEMVs + one MQA pass over the compressed cache.
- The 13.4GB of bf16 attention weights get requantized to 4-bit at load – ~3.4GB in VRAM.
- The 2.3GB embedding table is exiled to host RAM: one 28KB row gather per token.
- What's left buys **4 expert cache slots per layer** – and that has to be enough.

The pack: **repack, don't requantize**

- Moonshot ships K2.6 **quantization-aware-trained at int4**, group-32 – the weights are already 4-bit, and they were trained that way.
- Our converter repacks them **bit-exactly** into the paging format: 23,040 expert blobs, 4096-aligned, a layer's 384 experts contiguous on disk.
- 568GB checkpoint → 570.9GB pack + 23.4GB core, in about 25 minutes.

Zero added quality loss: the only conversion artifact is scale rounding – bf16 to f16, under one percent, on the scales alone. Every weight bit is Moonshot's own.

War story #1: the tokenizer **that lied**

- Kimi ships a **tiktoken** tokenizer – no `tokenizer.json` for standard Rust loaders.
- Built one via the official converter, patched 256 special tokens with explicit ids...
- ...and found `tokenizer_config.json` disagrees with the model's own tokenizer about `<|im_middle|>`: **163598 vs 163601**.
- Trust the tokenizer, not the config. Verified byte-exact against the reference on every test string.

Lesson: a wrong special-token id doesn't crash anything – it just quietly feeds the model a different conversation format than it was trained on.

War story #2: one sign bit, 570 gigabytes of blast radius

```
> What is a mixture-of-experts model?  
foss foss foss foss foss foss foss foss foss foss foss...
```

- First full run: pipeline perfect – 61 layers, 727GB streamed – output garbage.
- Root cause: compressed-tensors stores int4 as **offset-binary (value + 8)**, not two's complement. Every routed expert decoded as noise.
- My unit test "passed" – it validated against my own wrong assumption.
- Differential test against the official `unpack_from_int32` caught it in minutes. The fix made the repack a **verbatim nibble copy**.

Lesson: never validate an encoder against your own decoder. Always diff against the reference implementation.

First light – August 8, 2026

```
> What is a mixture-of-experts model? Answer in two sentences.
```

```
Key points to cover:
```

1. It's a machine learning architecture where multiple specialized sub-models (experts) are combined.
2. A gating network...

0.35

tok/s, stone-cold cache

2.1%

expert cache hit rate

829_{GB}

streamed for 60 tokens

Was it supposed to be that slow?

- Yes – that's the **disk-bound floor**, predicted at 0.5–1.5 tok/s before a line of Kimi code existed.
- Per token, worst case: 61 layers × 8 experts × 25MB ≈ **12GB of reads**. The disk does 7GB/s. Do the math.
- VRAM caches only 244 of 23,040 experts (1%); at that hit rate, every token is mostly disk.

Lever (all shipped)	Measured effect
Union prefill – fetch a chunk's experts once per layer	prompt cost capped
f16 KV cache	+14% decode, ½ memory
virtio iothreads at 25MB blobs	3.6 → 6.94 GB/s
VM RAM 64 → 128GB	enables the next slide

The **RAM tier**: a second cache between VRAM and disk

VRAM cache	244 experts	hit = free	
RAM tier	3,200 experts	hit = ~2ms memcpy	← new
NVMe pack	23,040 experts	miss = ~5ms contended read	

- 80GB of host memory as a managed expert cache – frequency-aware admission and eviction, at expert granularity, not the page cache's blind 4KB LRU.
- Every disk read write-allocates into the tier; hot experts stop touching the disk entirely.
- 13% of all experts fit – and routing is skewed, so effective hit rates run far higher.

One engine, **one API**, three model classes

Model	Weights	Speed	Role
Qwen3-30B-A3B	18.5 GB	41 tok/s	interactive chat
Qwen3-Coder-30B	18.5 GB	41 tok/s	interactive coding
Kimi K2.6 (1T)	594 GB	~1 tok/s	overnight agents, batch

- All three registered in one OpenAI-compatible server; request "model": "kimi-k2.6" and the registry swaps VRAM budgets automatically.
- Temperature, top-p, streaming, chat templates – per architecture, transparently.

In plain terms: the same box answers chat questions at reading speed, and thinks with a trillion-parameter model while you sleep.

What's next

- **Batched decode** – N agent streams share every expert fetch; aggregate throughput scales almost linearly at first.
- **Profiled pre-warm** – measure which experts your workload actually uses; preload the RAM tier before the first token.
- **Global VRAM slot pool** – let layers compete for cache instead of splitting it evenly.
- **Entropy-coded packs** – QAT int4 isn't uniform; 10–20% smaller is 10–20% faster when the disk is the wall.

The trajectory: 0.35 tok/s was first light. The measured levers above target 2+ tok/s single-stream, and multiples of that for batched agent workloads.

LLMPAGER

Model size is now a **disk-space problem**. Not a GPU-budget problem.

Apache-2.0 · Rust · CUDA · github.com/glennswest/llmpager

Standing on the shoulders of turbo-fieldfare – thanks for the idea that started this.